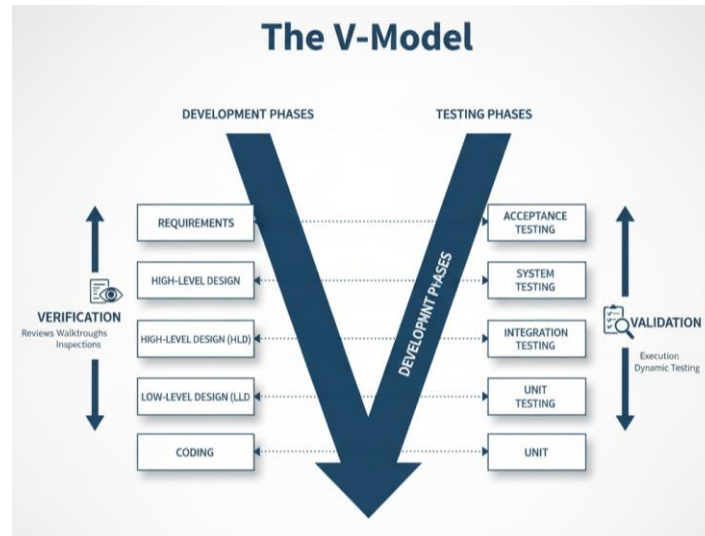


UNIT II: Essentials of Software Testing

Q1. Differentiate between Verification and Validation.

Verification and Validation are two key processes in software quality assurance.

- **Verification:** "Are we building the product **right**?" (Does it meet the technical specifications?)
- **Validation:** "Are we building the **right** product?" (Does it meet the user's needs?)



Basis of Difference	Verification	Validation
Main Goal	To check if the software is being built according to its design, requirements, and specifications.	To check if the final software meets the user's or client's business needs and expectations.
Focus	Process-Oriented. It focuses on documents, code, and design.	Product-Oriented. It focuses on the final, working product.
Timing	This is a static process. It is done <i>without</i> executing the code.	This is a dynamic process. It is done by <i>executing</i> the code.
Activities	Reviews, Walkthroughs, Inspections, Code Reviews, Document Reviews.	Functional Testing, System Testing, Usability Testing, User Acceptance Testing (UAT).
Performed by	Usually done by the QA team, developers, or peers.	Usually done by testers, end-users, and the client.
When it happens	Before validation. It happens <i>during</i> development (e.g., checking a design document before coding starts).	After verification. It happens <i>after</i> a feature or the whole product is built.
Error Found	Finds issues in logic, design, and documentation (e.g., a design that misses a requirement).	Finds bugs in the software's functionality (e.g., a "Submit" button that doesn't work).

Q2. What is Testing? Write down objective of Testing?

Software Testing is the process of evaluating a software application to find any bugs or defects. It involves executing the software and comparing its actual behavior with the expected behavior defined in the requirements.

The main purpose of testing is to ensure the software is high-quality, reliable, secure, and meets the needs of the user before it is released.

Objectives of Testing:

1. **To Find Defects (Bugs):** This is the primary objective. We test to find any errors, gaps, or missing requirements in the software.
2. **To Build Confidence:** By running tests and seeing them pass, the team gains confidence in the quality of the software and its readiness for release.
3. **To Verify Requirements:** Testing checks that the software meets all the functional and non-functional requirements that were agreed upon (e.g., "The user must be able to log in").
4. **To Prevent Defects:** By finding and fixing bugs early in the development cycle, testing helps prevent those bugs from causing bigger problems later.
5. **To Provide Information:** Testing provides valuable information to stakeholders (like project managers and clients) so they can make informed decisions, such as whether the software is ready to be launched.
6. **To Ensure User Satisfaction:** By finding and removing issues related to functionality and usability, testing helps ensure the final product is easy to use and works correctly for the end-user.

Q3. Explain Testing Life cycle.

The **Software Testing Life Cycle (STLC)** is a systematic, step-by-step process used by testing teams to plan, execute, and report on testing activities. It runs parallel to the Software Development Life Cycle (SDLC).



1. **Requirement Analysis:** The test team studies the software requirements (like Business Requirement Documents or User Stories) to understand *what* needs to be tested.
 - **Activity:** Identify testable requirements, clarify any doubts with clients or developers, and define the scope of testing.
2. **Test Planning:** This is a high-level phase where the test strategy is defined. A senior tester or test lead creates the **Test Plan** document.
 - **Activity:** Define test objectives, scope, resources (people, tools), schedule, and the "Exit Criteria" (when to stop testing).

3. **Test Case Design:** The testing team writes the detailed, step-by-step **Test Cases**.
 - **Activity:** Write test cases with steps, test data (e.g., 'user@test.com'), and expected results (e.g., "'Login Successful' message appears").
 4. **Test Environment Setup:** A dedicated environment (servers, hardware, software, data) is prepared specifically for the test team to run their tests.
 - **Activity:** Configure hardware and software, install the correct build of the application, and prepare test data.
 5. **Test Execution:** This is the phase where the testers actually run the software.
 - **Activity:** Testers execute the test cases written in Phase 3. They compare the *actual result* with the *expected result*. If they don't match, they report a **defect (bug)**.
 6. **Test Cycle Closure:** This is the final phase of the STLC, which occurs after testing is complete.
 - **Activity:** The team prepares a **Test Summary Report** that includes all testing metrics (e.g., total tests passed/failed, bugs found). They analyze the testing process to find lessons learned for the next project.
-

Q4. What is Test plan? Prepare sample test plan.

A **Test Plan** is a formal document that describes the **scope, objective, approach, resources, and schedule** of all testing activities. It is the central guiding document for the entire testing process.

It acts as a roadmap, helping the team understand *what* to test, *how* to test, *when* to test, and *who* will test. It is usually prepared by a Test Lead or Test Manager.

Sample Test Plan (Template) for a test plan for a new "E-commerce Website."

1. **Test Plan Identifier:** TP_Ecomm_v1.0
2. **Introduction:** This document outlines the test plan for the new E-commerce website (Project 'Zentara'), focusing on its initial launch features.
3. **Scope:**
 - **Features to be Tested:**
 - User Registration
 - User Login
 - Product Search
 - Add to Cart
 - **Features NOT to be Tested:**
 - Admin Panel
 - Payment Gateway (will be tested by a separate team)
 - Order History
4. **Test Strategy:**
 - We will perform **Functional Testing** to verify all features work as per requirements.
 - We will perform **Usability Testing** to check for ease of use.
 - We will perform **Compatibility Testing** on Chrome and Firefox browsers.
5. **Resources:**
 - **Test Team:** 2 Testers (Priya S., Rohan K.)
 - **Tools:** 'Bugzilla' for defect tracking.
6. **Schedule:**
 - **Start Date:** Nov 20, 2025
 - **End Date:** Dec 5, 2025
7. **Entry/Exit Criteria:**

- **Entry Criteria (When to start):**
 - The development build is stable.
 - All test cases are written and reviewed.
- **Exit Criteria (When to stop):**
 - 95% of all test cases have passed.
 - No "High" or "Critical" severity bugs are open.

8. **Deliverables (What we will produce):**

- Test Cases
 - Bug Reports
 - Test Summary Report
-

Q5. What is Test plan? How to execute test plan.

A **Test Plan** is a formal document that describes the **scope, objective, approach, resources, and schedule** of all testing activities. It is the central guiding document for the entire testing process.

It acts as a roadmap, helping the team understand *what* to test, *how* to test, *when* to test, and *who* will test.

How to Execute a Test Plan: You don't "execute" the test plan document itself. Instead, you "execute" the *activities* that are *defined* in the test plan.

"Executing the test plan" means putting the plan into action. The steps are:

1. **Meet Entry Criteria:** Before starting, the team must check that all conditions in the "Entry Criteria" section of the plan are met (e.g., the build is stable, the test environment is ready).
 2. **Team Kick-off:** The Test Lead shares the plan with the test team and other stakeholders (developers, managers) to ensure everyone understands their roles, the scope, and the schedule.
 3. **Monitor Test Environment Setup:** Ensure the environment is set up as specified in the plan.
 4. **Begin Test Execution:** Testers start running the test cases that were designed based on the plan's strategy.
 5. **Log Defects:** As bugs are found, they are reported and tracked using the tools and procedures defined in the plan.
 6. **Track Progress:** The Test Lead monitors the team's progress against the "Schedule" in the plan. They track metrics (e.g., tests passed vs. failed) to see if the project is on track.
 7. **Manage Risks:** If any "Risks" mentioned in the plan occur (e.g., a tester gets sick), the Test Lead implements the contingency plan.
 8. **Report Status:** The Test Lead provides regular status reports to management, as specified in the "Deliverables" section.
 9. **Check Exit Criteria:** Testing continues until the conditions in the "Exit Criteria" section are met (e.g., 95% of tests passed, no critical bugs).
 10. **Test Closure:** Once the exit criteria are met, the Test Lead prepares the final Test Summary Report and formally closes the testing cycle.
-

Q6. Define the term test plan, Test cases.

A **Test Plan** is a comprehensive, formal document that outlines the entire testing strategy for a project. It is a high-level document that answers "who, what, when, where, and how" of testing. It is created *before* testing begins and guides the entire effort.

It typically includes:

- Scope (what will be tested and what won't)
- Objectives
- Schedule
- Resources (people, tools)
- Test Strategy
- Entry and Exit Criteria (when to start and stop testing)
- Risks and Contingencies

A **Test Case** is a detailed, low-level document that describes a specific test to be performed on the software. It is a set of step-by-step instructions to verify a single piece of functionality. Many test cases are written to cover the scope defined in the test plan.

It typically includes:

- **Test Case ID:** A unique identifier (e.g., TC-001).
- **Test Scenario:** The feature being tested (e.g., "Verify valid user login").
- **Test Steps:** The exact instructions to follow (e.g., "1. Open browser. 2. Enter URL. 3. Enter username...").
- **Test Data:** The specific data to use (e.g., Username: 'test', Password: '123').
- **Expected Result:** What *should* happen (e.g., "User is redirected to the dashboard").
- **Actual Result:** What *actually* happened when the test was run.
- **Status:** Pass / Fail.

Q7. With the help of case studies write down Test plan and Test cases.

Case Study: Testing a "Forgot Password" Feature

1. Mini Test Plan

- **Project:** Website "Forgot Password" Feature
- **Scope:**
 - **In Scope:** Verifying that a user who forgot their password can successfully receive a reset link via email and set a new password.
 - **Out of Scope:** The login page itself, user registration.
- **Test Strategy:** We will perform **Functional Testing** (positive and negative) to ensure the feature works as expected. We will also perform basic **Security Testing** to ensure the reset link is not predictable.
- **Resources:** 1 Tester, 1 test email account.
- **Exit Criteria:**
 - User can successfully reset their password.
 - User with an invalid email is blocked.
 - All high-priority test cases are passed.

2. Test Cases

ID	Test Scenario	Test Steps	Test Data	Expected Result
TC_FP_01	Positive Test: Valid Email	1. Go to Login page. 2. Click "Forgot	Email: valid_user@test.com	1. A success message "Password reset link sent" is

		Password". 3. Enter a valid, registered email. 4. Click "Submit".		shown. 2. The user receives an email with a reset link.
TC_FP_02	Negative Test: Invalid Email	1. Go to Login page. 2. Click "Forgot Password". 3. Enter an email that is <i>not</i> registered. 4. Click "Submit".	Email: invalid_user@test.com	An error message "Email not found" is shown. (Note: For security, some sites just say "If email exists..." which is also a valid expected result).
TC_FP_03	Negative Test: Blank Email	1. Go to Login page. 2. Click "Forgot Password". 3. Leave the email field blank. 4. Click "Submit".	Email: (empty)	An error message "Email is required" is shown below the field.
TC_FP_04	End-to-End Flow: Reset Password	1. Perform all steps in TC_FP_01. 2. Open the email and click the reset link. 3. Enter a new password in both fields. 4. Click "Save". 5. Try to log in with the <i>new</i> password.	New Pass: NewPass123	1. User sees "Password reset successful". 2. User is able to log in successfully with NewPass123.

Q8. Write note on a) TDD b) BDD c) ATDD

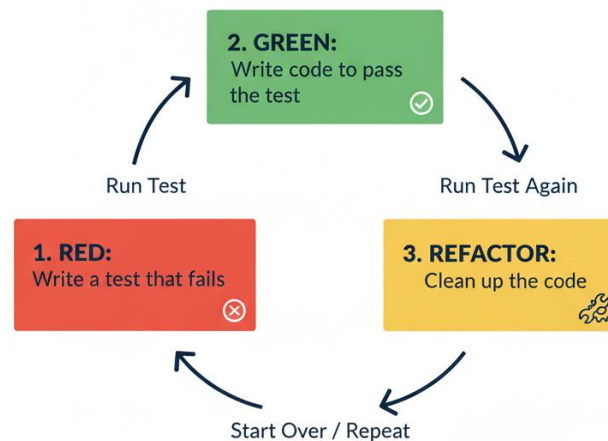
a) TDD (Test-Driven Development)

TDD is a software development process where developers write tests *before* they write the actual production code. It is a "developer-focused" technique.

The process follows a short, repetitive cycle called **Red-Green-Refactor**:

1. **RED:** The developer writes a small, automated **unit test** for a new feature. This test will *fail* (turn "Red") because the feature's code doesn't exist yet.
2. **GREEN:** The developer writes the *minimum amount of production code* possible, just enough to make the new test pass (turn "Green").
3. **REFACTOR:** The developer now cleans up the code (improves its design, removes duplication) while making sure all tests still pass.

Benefit: TDD results in high-quality, clean code with a comprehensive "safety net" of unit tests, making the code base safer to change and maintain.



b) BDD (Behavior-Driven Development)

BDD is an Agile development process that improves communication between technical (developers, QAs) and non-technical (Business Analysts, clients) team members.

It focuses on describing the *behavior* of the system from the user's perspective. It uses a simple, English-like language called **Gherkin**.

The Gherkin syntax uses **Given-When-Then** keywords:

- **Given:** The initial context (e.g., Given I am on the login page)
- **When:** The action the user performs (e.g., When I enter valid credentials)
- **Then:** The expected outcome (e.g., Then I should be logged in)

Benefit: BDD ensures that the entire team has a shared understanding of what needs to be built *before* development starts, reducing misunderstandings.

c) ATDD (Acceptance Test-Driven Development)

ATDD is a collaborative development process that is very similar to BDD. It brings together the "three amigos": the **customer** (or BA), the **developer**, and the **tester**.

This team discusses a new feature (like a User Story) and together they define the **acceptance criteria** (the conditions that must be met for the feature to be considered "done").

These acceptance criteria are written as "Acceptance Tests" *before* development begins. The developers then write code to make these acceptance tests pass.

Difference from BDD: While very similar, ATDD's primary focus is on the *collaborative process* of defining acceptance criteria. BDD is more focused on the *language and format* (Gherkin) used to describe behavior. ATDD is the "why" (collaboration) and BDD is often the "how" (language).